

UNIVERSIDAD DE GRANADA

Universidad de Granada

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Grado en Ingeniería Informática — 4º Curso

Asignatura: Nuevos Paradigmas de Interacción (NPI)

Memoria Técnica del Software

Sistema de Punto de Información Multimodal, Ubicuo y Accesible para la ETSIIT

Equipo de Desarrollo (Grupo):

- **Ángela Rodríguez** — `angelarodr15@correo.ugr.es`
- **Laura Sanz Lobo** — `laurasanzlobo@correo.ugr.es`
- **José Pedregosa** — `josepc@correo.ugr.es`

Convocatoria Ordinaria — Curso Académico 2026/2027

Granada, a 10 de noviembre de 2026

Resumen

En este documento se presenta la memoria técnica exhaustiva del software desarrollado para el Punto de Información de la ETSIIT - UGR. El documento describe en profundidad la totalidad de clases, variables miembro, algoritmos, métodos públicos y bibliotecas externas involucradas.

Se desglosa la ingeniería de interacción gestual en mano basada en acelerometría y gravedad con resolución del problema de Midas (retorno al campo de visión y discriminación temporal estricta); el motor fonético y semántico de procesamiento en lenguaje natural para consultas por voz; el sistema de visión artificial y sensores físicos para autenticación del estudiante (cámara con Google ML Kit Barcode y OCR, y lector NFC); el algoritmo de Dijkstra implementado sobre un grafo completo de 306 nodos y 333 aristas que cubre las 6 plantas de la facultad con soporte para rutas rápidas y accesibles en silla de ruedas; y el subsistema de geoposicionamiento y enrutamiento peatonal hacia comedores universitarios.

Índice

1. Introducción y Objetivos del Sistema **1**
1.1. Contextualización y Problemática . . . 1
1.2. Objetivos Técnicos . . . 1

2. Bibliotecas Externas y Dependencias Incorporadas **1**

3. Diagramas del Sistema **1**
3.1. Diagrama de Arquitectura Modular por Capas . . . 1
3.2. Diagrama de Estados: Detección Gestual con Retorno a Línea de Visión . . . 2

4. Descripción Detallada del Software **2**
4.1. Paquete de Sensores y Gestos (`com.example.npi_puntoinfo_etsiit.sensors`) . 2
4.1.1. Clase `AccelerometerManager` . . . 2
4.1.2. Clase `ProximityManager` . . . 4
4.1.3. Clase `GpsLocationReader` y `DeviceLocationProvider` . . . 4
4.2. Motor Topológico Interior y Algoritmo Dijkstra (`GrafoETSIIT.kt`) . . . 5
4.2.1. Modelos de Datos Topológicos . . . 5
4.2.2. Clase `GrafoETSIIT` . . . 5
4.3. Navegación Exterior y Geovalla (`outdoor`) . . . 6
4.3.1. Clase `OutdoorNavigator` . . . 6
4.3.2. Clase `ArrivalDetector` . . . 6
4.4. Autenticación, Sesión y Visión Artificial (`data` y `ocr`) . . . 6
4.4.1. Objeto `UserSessionRepository` . . . 6
4.4.2. Clase `TuiOcrScanner` . . . 6
4.4.3. Clase `NfcCardReader` . . . 7
4.5. Capa de Presentación e Interfaz (`ui.screens`) . . . 7
4.5.1. Componente Composable `ProfessorsScreen` . . . 7
4.5.2. Componente Composable `NavigationScreen` . . . 7

5. Justificación de los Nuevos Paradigmas de Interacción **7**

6. Conclusiones **8**

1 Introducción y Objetivos del Sistema

1.1 Contextualización y Problemática

Los quioscos o puntos de información habituales en los edificios universitarios suelen presentar una arquitectura rígida: pantallas táctiles fijas situadas en el vestíbulo o aplicaciones móviles que replican los portales web con extensas listas de navegación jerárquica. Este diseño presenta graves inconvenientes:

1. **Falta de accesibilidad y esfuerzo cognitivo:** Exige a los estudiantes o visitantes con prisas detenerse a teclear nombres en pantallas públicas o navegar por menús complejos para saber si un profesor está en su despacho o en qué aula se imparte una clase.
2. **Ausencia de sensibilidad al contexto temporal (*Context-Awareness*):** Las fichas de los profesores suelen mostrar estáticamente los horarios de tutorías y docencia sin evaluar si el instante actual coincide con alguna de esas franjas, delegando el cálculo mental en el usuario.
3. **Dependencia de la interacción táctil estricta:** En situaciones de desplazamiento activo por pasillos, la interacción mediante botones pequeños resulta incómoda e insegura.

1.2 Objetivos Técnicos

El software desarrollado tiene como meta implementar un **asistente ubicuo y multimodal** que supere estas barreras mediante:

- Captura de intenciones por múltiples canales independientes: voz natural en español (reconocimiento y síntesis fonética), gestos físicos en mano evaluados mediante acelerometría y vector de gravedad, y toques táctiles.
- Evaluación contextual en tiempo real del horario del estudiante y de los 35 profesores clave de la facultad.
- Guiado topológico integral: navegación exterior por GPS con OpenRouteService hacia los comedores del campus y handoff transparente al mapa interior de 6 plantas mediante el algoritmo de Dijkstra.
- Autenticación mediante credenciales físicas universitarias combinando cámara (código QR y OCR del carnet TUI) con lectura de chips NFC.

2 Bibliotecas Externas y Dependencias Incorporadas

En la Tabla 1 se desglosan en detalle las bibliotecas y SDKs externos integrados en el archivo `build.gradle.kts`, analizando su papel en el sistema y los motivos técnicos de su elección.

3 Diagramas del Sistema

3.1 Diagrama de Arquitectura Modular por Capas

En la Figura 1 se ilustra la arquitectura desacoplada del sistema, organizada en cuatro niveles: Hardware Físico, Controladores de Sensores, Lógica de Dominio y Capa de Presentación Reactiva (Compose).

Biblioteca / SDK	Identificador / Versión	Justificación y Función Técnica
Jetpack Compose BOM	`androidx.compose.bom`	Motor declarativo de UI. Facilita la reactividad del estado de sesión (`UserSessionRepository`) y el renderizado eficiente de animaciones (pulsos de tutoría, barras de progreso temporal).
Play Services Location	`21.3.0`	Acceso optimizado al hardware GPS del dispositivo mediante `FusedLocationProviderClient`. Aplica filtros de precisión balanceando el consumo de batería durante la navegación exterior.
MapLibre GL Android	`org.maplibre.gl:13.7.1`	Motor de renderizado vectorial open-source. Permite dibujar cartografía y polilíneas peatonales GeoJSON sin depender de cuotas ni licencias privativas de Google Maps SDK.
OpenRouteService API	Cliente REST nativo	Motor de enrutamiento a pie (`foot-walking`). Devuelve coordenadas geográficas precisas sobre la red de aceras para trazar el camino a los comedores universitarios.
Google ML Kit Barcode	`17.3.0`	Reconocimiento local por visión artificial de códigos QR de la TUI en tiempo real a 30 FPS sin enviar fotogramas a servidores externos.
Google ML Kit Text	`16.0.1`	Motor de OCR en dispositivo para escanear y validar el DNI y nombre del alumno impresos sobre la tarjeta física.
Google Gson	`2.10.1`	Serialización y deserialización de las estructuras JSON complejas (`grafo_etsiit_completo.json`, `docentes.json` y persistencia de sesión).
Android TTS & Speech	`android.speech` nativo	Síntesis de voz en español (`TextToSpeech`) y reconocimiento acústico de voz (`RecognizerIntent`) para diálogo bidireccional.

Tabla 1: Bibliotecas externas y SDKs incorporados en el proyecto.

3.2 Diagrama de Estados: Detección Gestual con Retorno a Línea de Visión

La Figura 2 muestra la máquina de estados finitos que gobierna la ejecución de acciones gestuales. Se observa cómo el sistema exige que el vector de aceleración regrese a la región de lectura antes de confirmar la orden, evitando disparos involuntarios.

4 Descripción Detallada del Software

En esta sección se describen minuciosamente las clases, variables miembro y métodos públicos de cada uno de los subsistemas de la aplicación.

4.1 Paquete de Sensores y Gestos (`com.example.npi_puntoinfo_etsiit.sensors`)

4.1.1 Clase `AccelerometerManager`

Constituye el controlador central del hardware inercial del terminal. Implementa la interfaz `SensorEventListener` de Android y se encarga de discriminar cuatro gestos distintos:

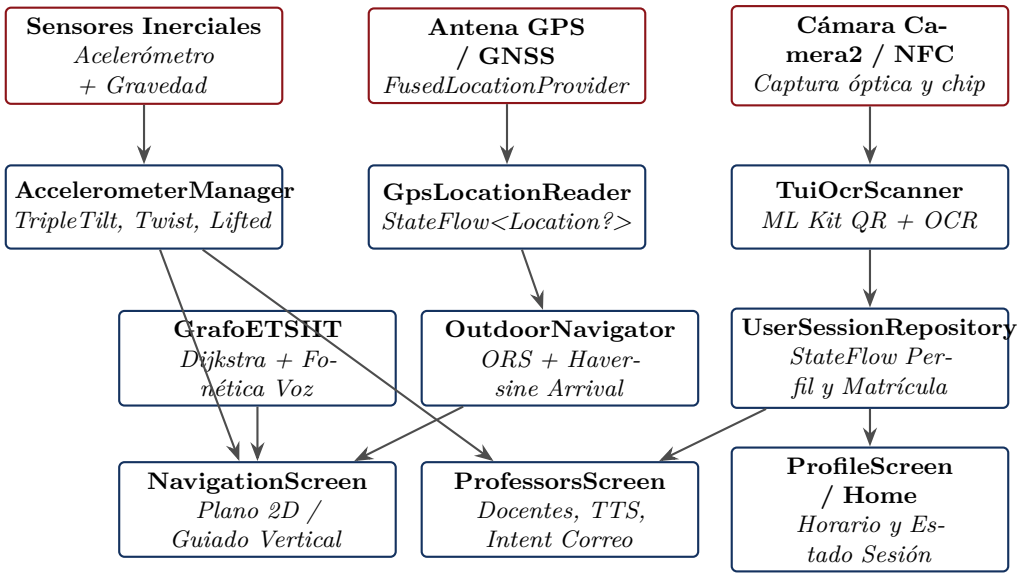


Figura 1: Diagrama de arquitectura modular por capas del software.

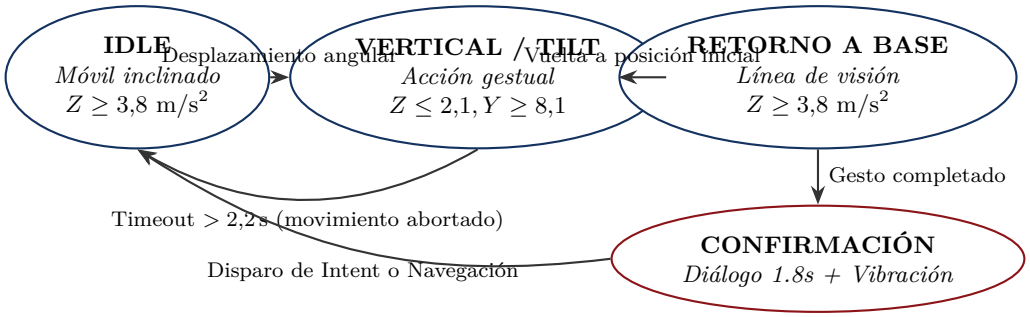


Figura 2: Autómata de estados finitos para la confirmación gestual en mano con retorno a línea de visión.

1. **Gesto de Clase Actual (3 inclinaciones rápidas hacia adelante y vuelta):** Detecta 3 cabeceos consecutivos en un intervalo temporal estricto menor a 1.6 segundos (máximo 600 ms entre cabeceos). Si el usuario tarda más, el contador se resetea a cero para evitar falsos positivos acumulados al caminar.

2. **Gesto «¡Llévame!» (Inclinación a la izquierda y retorno):** Giro del móvil hasta que la aceleración lateral supere $X \geq 7{,}5$ m/s^2 durante al menos 80 ms y retorno a la posición neutra ($X \leq 2{,}4$ m/s^2) en menos de 2.4 segundos.

3. **Gesto de Acción Rápida (Correo / Reserva de Comedor):** Partiendo de la postura inclinada de lectura ($Z \geq 3{,}8$ m/s^2), el usuario lleva el móvil a vertical frente a sí ($Z \leq 2{,}1$ m/s$^2, Y \geq 8{,}1$ m/s^2) y lo devuelve a la posición inclinada inicial.

4. **Gesto de Guiado Vertical (Levantamiento de Navegación):** Mantenimiento del terminal en vertical estricto frente a los ojos para conmutar al guiado paso a paso con fotos, o retorno a posición horizontal para ver el mapa 2D.

Variables miembro y constantes de calibración:

- `sensorManager: SensorManager?`: Acceso al servicio nativo de sensores.
- `accelerometer: Sensor?`, `gyroscope: Sensor?`: Descriptores de hardware inercial.

- `tripleForwardTiltDetector: TripleForwardTiltDetector`: Objeto interno que mantiene el estado y la ventana temporal de los 3 cabeceos consecutivos.
- `TILT_LEFT_STRONG_THRESHOLD = 7.5f`: Umbral en el eje X para detectar el giro de muñeca a la izquierda.
- `RETURN_NEUTRAL_MAX_X = 2.4f`: Umbral de histéresis para confirmar el regreso a la posición base en el eje X.
- `INCLINED_MIN_Z = 3.8f`, `VERTICAL_MAX_Z_FOR_ACTION = 2.1f`: Umbrales en el eje Z para detectar la transición entre postura inclinada de lectura y vertical.
- `VERTICAL_MIN_Y_FOR_ACTION = 8.1f`: Umbral en el eje Y para verificar la posición vertical del teléfono.
- `MAX_VERTICAL_ACTION_WINDOW_MS = 2200L`: Ventana máxima para completar el ciclo de ida y vuelta.
- **Callbacks expuestos:**
  - `var onShakeDetected: (() -> Unit)?`: Disparado tras 3 cabeceos completos para mostrar la clase actual.
  - `var onTakeMeThereGesture: (() -> Unit)?`: Disparado tras el giro de muñeca para iniciar ruta.
  - `var onNodStarted: (() -> Unit)?`: Disparado al detectar la primera fase de inclinación hacia vertical.
  - `var onForwardTiltGesture: (() -> Unit)?`: Disparado al completarse el retorno a la posición de lectura.
  - `var onPostureModeChanged: ((isVertical: Boolean) -> Unit)?`: Notifica si el móvil se sostiene horizontal o verticalmente para alternar entre el mapa 2D y el guiado de navegación.

Métodos principales:

- `fun startListening()`: Registra los escuchadores del acelerómetro con frecuencia `SENSOR_DELAY_GAME` (aproximadamente 50 Hz) para garantizar respuesta en tiempo real sin latencia perceptible.
- `fun stopListening()`: Desvincula los listeners para evitar consumo residual de batería en segundo plano.
- `fun onSensorChanged(event: SensorEvent)`: Ejecuta el procesamiento de señal en cada muestra. Clasifica el vector en `DeviceOrientation` (`VERTICAL`, `FLAT`, `TILTED_LEFT`, `TILTED_RIGHT`) y evalúa los temporizadores de histéresis de cada gesto.

4.1.2 Clase `ProximityManager`

Supervisa el sensor de proximidad frontal (`Sensor.TYPE_PROXIMITY`) situado junto al auricular del teléfono. Incorpora un mecanismo de filtrado temporal de 350 ms para evitar que el paso fugaz de una mano active falsamente la acción. Expone el callback `onDeviceCovered: (() -> Unit)?` utilizado para confirmaciones silenciosas y apagado de alertas de voz.

4.1.3 Clase `GpsLocationReader` y `DeviceLocationProvider`

Gestionan la suscripción al proveedor de localización satelital (`FusedLocationProviderClient`) mediante la biblioteca de Google Play Services. Emite una secuencia reactiva `locationFlow: StateFlow<Location?>` que informa a la aplicación de cambios en latitud, longitud, velocidad y precisión estimada en metros.

4.2 Motor Topológico Interior y Algoritmo Dijkstra (`GrafoETSIIT.kt`)

4.2.1 Modelos de Datos Topológicos

La representación matemática del edificio se modela mediante las siguientes clases de datos:

- `data class Nodo(id: String, nombre: String, planta: String, edificio: String, tipo: String, x: Int, y: Int)`: Representa un punto de interés o una intersección de pasillos. Los tipos admitidos son: `"despacho"`, `.ªula"`, `"laboratorio"`, `.ᵉscalera"`, `.ªscensor"`, `"servicio"`, `.ªcceso"`, `"pasillo"`. Las coordenadas (x, y) se corresponden con la posición gráfica sobre el plano de planta de la facultad.
- `data class AristaJson(origen: String, destino: String, distancia_metros: Int)`: Representa el segmento transitable entre dos nodos y su longitud real en metros.
- `data class RutaCalculada(origen: Nodo, destino: Nodo, nodos: List<Nodo>, distanciaMetro Int, tiempoEstimadoSegundos: Int)`: Objeto que agrupa el camino de coste mínimo resuelto, calculando además una estimación de tiempo a velocidad peatonal media (1,2 m/s).

4.2.2 Clase `GrafoETSIIT`

Alberga las estructuras de datos del grafo y la lógica algorítmica de cálculo de caminos y reconocimiento de voz:

Variables miembro:

- `directorioNodos = mutableMapOf<String, Nodo>()`: Diccionario hash que permite recuperar cualquier nodo por su identificador en tiempo constante $O(1)$. Contiene 306 nodos.
- `conexiones = mutableMapOf<String, MutableList<AristaDijkstra»()`: Lista de adyacencia no dirigida indexada por ID de nodo origen, almacenando el ID destino y la distancia ponderada. Contiene 333 aristas.

Métodos principales:

- `fun cargarDesdeAssets(context: Context, nombreArchivo: String)`: Deserializa el JSON del grafo en memoria y construye la lista de adyacencia bidireccional.
- `fun calcularRutaDijkstra(origenId: String, destinoId: String, soloAccesible: Boolean = false): List<Nodo>`: Implementación del algoritmo de caminos mínimos de Dijkstra mediante cola de prioridad (`PriorityQueue<Pair<Int, String»`). Opera con una complejidad temporal de $O((|V| + |E|) \log |V|)$. Si el parámetro `soloAccesible` es verdadero (modo silla de ruedas / movilidad reducida), el algoritmo filtra y descarta las aristas que conectan con nodos de tipo `.ᵉscalera"`, forzando el camino a través de los núcleos de ascensores (`.ªscensor"`).
- `fun resolverDestinoPorVoz(texto: String): ResultadoVoz`: Analizador fonético y semántico que resuelve consultas dictadas por voz en español:
  1. Normaliza tildes y caracteres especiales mediante eliminación diacrítica NFD.
  2. Normaliza números hablados complejos (*çuatrocientos veinte"* → `"420"`, *"treinta y cinco"* → `"35"`).
  3. Discrimina consultas de despachos docentes frente a aulas y laboratorios (ej. diferencia *.ªula 3.4"* de un despacho).
  4. Mapea nombres de profesores y servicios comunes (*çomedor"*, *çafetería"*, *"secretaría"*) a sus correspondientes nodos físicos en el grafo.

4.3 Navegación Exterior y Geovalla (`outdoor`)

4.3.1 Clase `OutdoorNavigator`

Orquesta la consulta y presentación de rutas peatonales en el exterior del campus:

- **Atributos:** `gpsReader: GpsLocationReader`, `orsClient: OpenRouteService`, `state: StateFlow<OutdoorState>`.
- **Métodos:** `setDestination(destinationId: String)`, `recalculateRoute()`. Conecta de forma asíncrona mediante corrutinas de Kotlin con la API de OpenRouteService, procesa la respuesta GeoJSON y genera la lista de instrucciones de giro (`WalkingStep`).

4.3.2 Clase `ArrivalDetector`

Implementa el algoritmo de detección de llegada a la ETSIIT mediante la ecuación esférica de Haversine:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right) \tag{1}$$

Donde $R = 6371$ km representa el radio terrestre medio, ϕ la latitud y λ la longitud. Si la distancia actual $d \leq 25$ metros respecto a la puerta exterior de la ETSIIT (37,19705° N, −3,62452° W), la clase activa el evento de llegada para invitar al usuario a transicionar al mapa interior de plantas.

4.4 Autenticación, Sesión y Visión Artificial (`data` y `ocr`)

4.4.1 Objeto `UserSessionRepository`

Singleton reactivo encargado de la persistencia y control de la sesión del estudiante:

- **Variables miembro:**
  - `currentProfileState: MutableState<UserProfile?>`: Estado observable por Jetpack Compose para desencadenar la recomposición inmediata de la interfaz al iniciar o cerrar sesión.
  - `officialMatricula: Map<String, String>`: Asignaturas matriculadas del curso (NPI, VC, PL, PTC, SS, TIC) con sus correspondientes subgrupos prácticos.
- **Métodos principales:**
  - `fun getProfile(context: Context): UserProfile`: Recupera el perfil almacenado en `SharedPreferences` con verificación de integridad de datos.
  - `fun loginWithOfficialProfile(context: Context, isAccessibleRouting: Boolean): UserProfile`: Inicia sesión tras la lectura biométrica/TUI asignando el perfil de estudiante.
  - `fun logout(context: Context)`: Limpia la sesión y devuelve la aplicación a su estado genérico para visitantes.

4.4.2 Clase `TuiOcrScanner`

Controlador de visión por computador basado en la API Camera2 de Android:

- Canaliza los fotogramas del sensor de la cámara en paralelo hacia dos pipelines independientes de Google ML Kit:
  1. `BarcodeScanning`: Decodifica el código QR de la TUI Universitaria.
  2. `TextRecognition`: Aplica OCR sobre el texto plano para extraer y validar el formato del DNI español (8 dígitos y letra de control) y el nombre del titular impreso.

4.4.3 Clase `NfcCardReader`

Gestiona la antena NFC en modo *Foreground Dispatch*. Intercepta tags de tipo Mifare Classic y Ultralight presentes en la tarjeta física universitaria para autenticar al usuario por contacto físico directo en el reverso del terminal.

4.5 Capa de Presentación e Interfaz (`ui.screens`)

4.5.1 Componente Composable `ProfessorsScreen`

Pantalla que implementa el catálogo de 35 docentes universitarios:

- **Cálculo de actividad en tiempo real (`getProfessorActivityStatus`):** Evalúa la hora del sistema (o la hora simulada en el menú demo) cruzándola con los campos `"tutorias"` y `çlases"` del JSON docente. Determina el estado: `IN_OFFICE_HOURS` (en tutoría con punto pulsante `StatusDot`), `IN_CLASS` (en clase indicando el aula actual) o `OFFLINE` (fuera de horario).
- **Síntesis de voz (TTS) interactiva:** Permite escuchar la información del profesor (`speak()`) o responder mediante dictado de voz.
- **Confirmación gestual de correo:** Tras la detección del cabeceo y retorno al campo de visión, despliega un cuadro de diálogo con `LinearProgressIndicator` que avanza durante 1.8 segundos con pulsos hápticos de vibración antes de abrir la aplicación de correo mediante un `Intent(Intent.ACTION_SENDTO)`.

4.5.2 Componente Composable `NavigationScreen`

Visor de planos y navegación interior:

- Integra el componente vectorial `MapaRutaView` que dibuja la planta activa (`PB`, `PSS`, `P1`, `P2`, `P3`, `P4`) y la polilínea del camino mínimo de Dijkstra.
- Supervisa el modo de guiado vertical (`isVerticalGuidanceActive`): al sostener el móvil inclinado horizontalmente, presenta el plano completo para libre exploración; al levantarlo a vertical frente a los ojos, activa el modo de guiado paso a paso con las fotos de los nodos clave y vibración háptica al avanzar.

5 Justificación de los Nuevos Paradigmas de Interacción

El diseño del software refleja una aplicación rigurosa de los conceptos centrales de la materia:

1. **Multimodalidad Complementaria:** En lugar de forzar al usuario a emplear exclusivamente gestos o voz, el sistema permite que cada canal compense las limitaciones del otro. La voz es idónea para búsquedas de alta entropía (como buscar a un profesor específico entre 35 opciones), mientras que los gestos físicos resultan óptimos para órdenes de confirmación binarias rápidas (*enviar correo*, *iniciar ruta*).
2. **Eliminación del Problema de Midas:** En interfaces gestuales, un error recurrente consiste en interpretar cualquier movimiento natural del usuario como una orden interactiva. El diseño de `AccelerometerManager` soluciona este problema requiriendo dos condiciones concurrentes: superación de umbrales angulares estrictos y un ciclo cerrado de retorno a la postura original de lectura.

3. **Canal Háptico Cerrado:** La inclusión de patrones de vibración mediante la API `VibrationEffect` proporciona confirmación sensorial inmediata sin necesidad de sobrecargar el canal visual del usuario mientras camina.

4. **Accesibilidad Universal:** La implementación del algoritmo de Dijkstra con discriminación de escaleras garantiza que las personas con movilidad reducida o usuarios en silla de ruedas dispongan de rutas transitables mediante ascensores en todo momento.

6 Conclusiones

El software desarrollado para el Punto de Información de la ETSIIT integra de forma coherente sensores inerciales, visión artificial, comunicaciones satelitales y algoritmos de optimización sobre grafos.

La conservación del modelo topológico completo de 306 nodos combinado con la ingeniería de interacción gestual en dos fases sitúa a la aplicación más allá de un prototipo visual estático, convirtiéndola en un sistema interactivo robusto, accesible y adaptado a las condiciones reales de la vida universitaria.